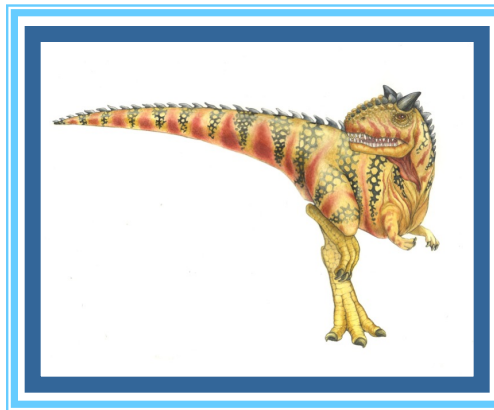


Chapter 3: Inter Process Communication (IPC)





Interprocess Communication

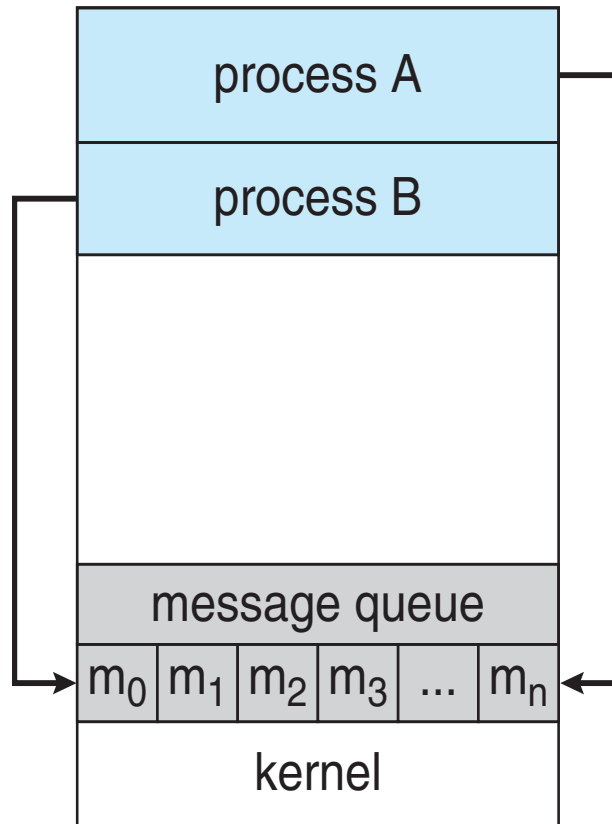
- Processes within a system may be *independent* or *cooperating*
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
 - Convenience
- Cooperating processes need **interprocess communication (IPC)**
- Two models of IPC
 - **Shared memory**
 - **Message passing**



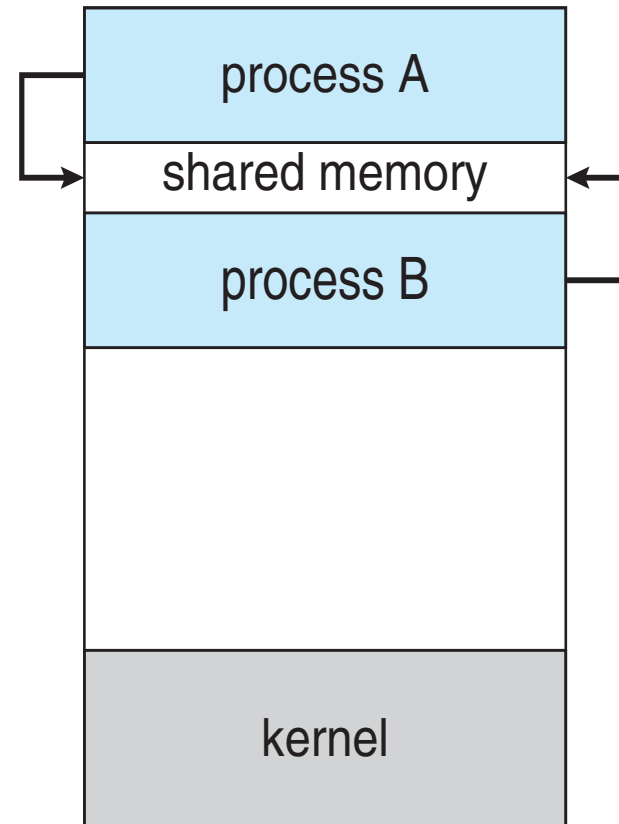


Communications Models

(a) Message passing. (b) shared memory.



(a)



(b)





Interprocess Communication – Shared Memory

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.
- *Synchronization* is needed, which will be discussed in great details in Chapter 5.





Interprocess Communication – Message Passing

- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
 - **send**(*message*)
 - **receive**(*message*)
- The *message* size is either fixed or variable





Message Passing (Cont.)

- If processes P and Q wish to communicate, they need to:
 - Establish a **communication link** between them
 - Exchange messages via send/receive
- Implementation of communication link
 - Physical:
 - ▶ Shared memory
 - ▶ Hardware bus
 - ▶ Network
 - Logical:
 - ▶ Direct or indirect
 - ▶ Synchronous or asynchronous
 - ▶ Automatic or explicit buffering





Direct Communication

- Processes must name each other explicitly:
 - **send** (P , *message*) – send a message to process P
 - **receive**(Q , *message*) – receive a message from process Q
- Properties of communication link
 - Links are established automatically
 - A link is associated with exactly one pair of communicating processes
 - Between each pair there exists exactly one link
 - The link may be unidirectional, but is usually bi-directional





Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
 - Each mailbox has a unique id
 - Processes can communicate only if they share a mailbox
- Properties of communication link
 - Link established only if processes share a common mailbox
 - A link may be associated with many processes
 - Each pair of processes may share several communication links
- Operations
 - create a new mailbox (port)
 - send and receive messages through mailbox
 - destroy a mailbox
- Primitives are defined as:
send(*A, message*) – send a message to mailbox *A*
receive(*A, message*) – receive a message from mailbox *A*





Indirect Communication

■ Mailbox sharing

- P_1 , P_2 , and P_3 share mailbox A
- P_1 , sends; P_2 and P_3 receive
- Who gets the message?

■ Solutions

- Allow a link to be associated with at most two processes
- Allow only one process at a time to execute a receive operation
- Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.





Communications in Client-Server Systems

- Pipes
- Sockets
- Remote Procedure Calls
- Remote Method Invocation (Java)





Pipes

- Acts as a conduit allowing two processes to communicate
- Questions:
 - Is communication unidirectional or bidirectional?
 - Must there exist a relationship (i.e., ***parent-child***) between the communicating processes?
- Ordinary pipes – cannot be accessed from outside the process that created it. Typically, a parent process creates a pipe and uses it to communicate with a child process that it created.
- Named pipes – can be accessed without a parent-child relationship.





Pipes

* Pipe

■ A simple, **unnamed** pipe provides a one-way flow of data.

- Can be thought as a special file that can store a limited amount of data in a first-in-first-out manner, exactly akin to a queue.

■ Other variations:

- ▶ Stream pipes
- ▶ FIFOs

■ An unnamed pipe is created by calling ***pipe()***, which returns an array of 2 file descriptors (int).

- The file descriptors are for reading and writing, respectively





pipe System Call (unnamed)

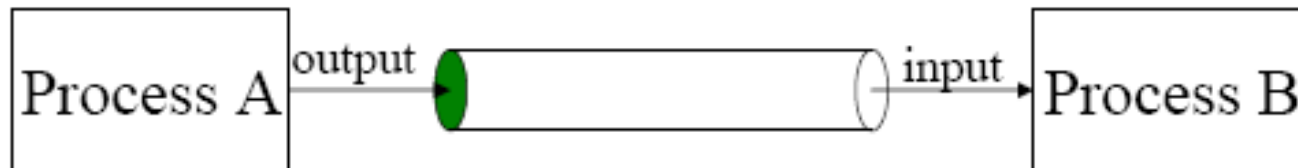
Creates a half-duplex pipe.

- Include(s): <unistd.h>
- Syntax: ***int pipe (int pipefd[2]);***
- Return: Success: 0; Failure: -1; Sets errno: Yes
 - What does it mean to return errno?
- Arguments: None
- If successful, the ***pipe*** system call will return two integer file descriptors, pipefd[0] and pipefd[1].
 - pipefd[1] is the write end to the pipe.
 - pipefd[0] is the read end from the pipe.
- Parent/child processes communicating via unnamed pipe.





Creating a Pipe



- Pipe is a communication channel abstraction
 - Process A can write to one end using “write” system call
 - Process B can read from the other end using “read” system call
- System call

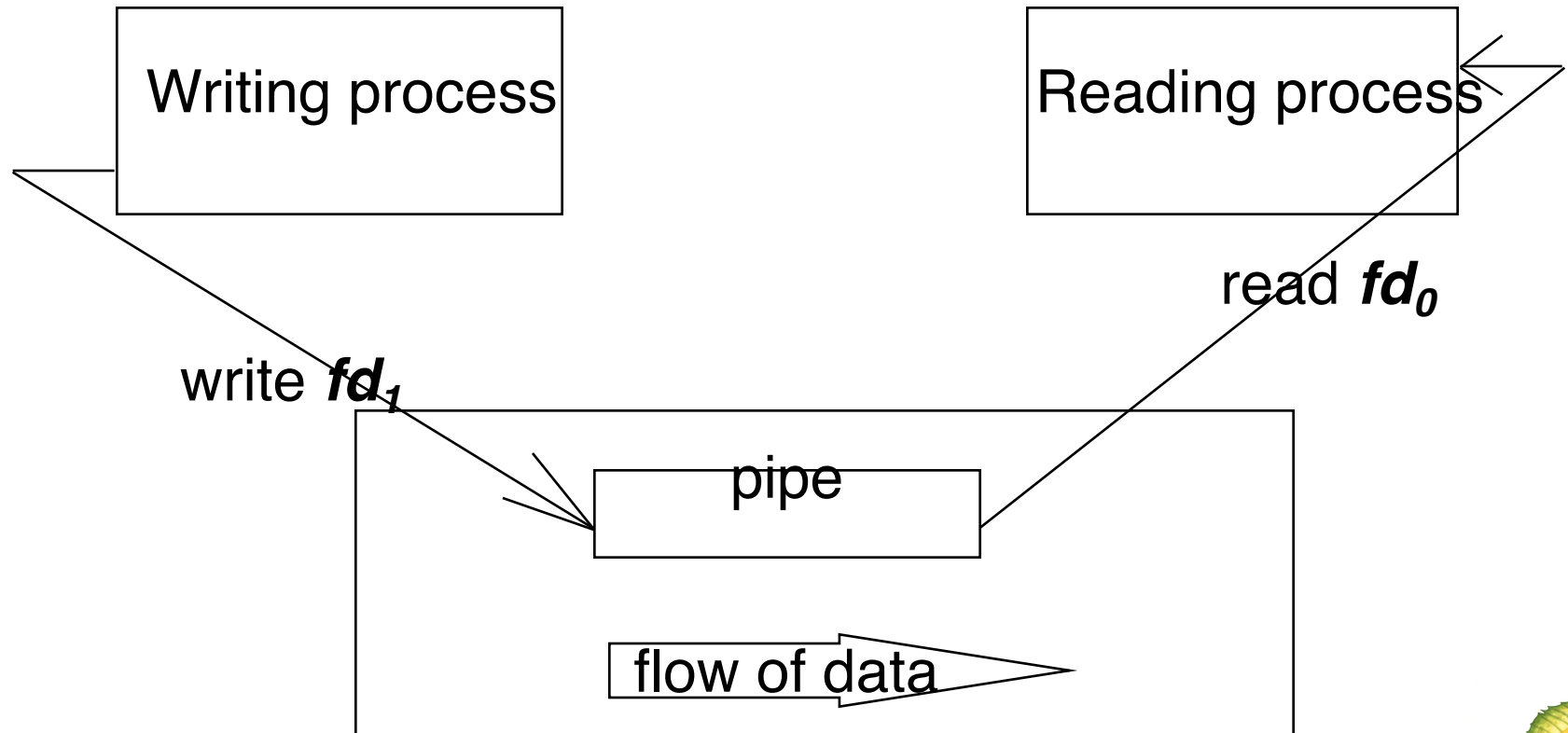
```
int pipe( int fd[2] );  
return 0 upon success -1 upon failure  
fd[0] is open for reading  
fd[1] is open for writing
```
- Two coordinated processes created by `fork` can pass data to each other using a pipe.





Piping Between Two Processes

- The pipe is represented in an array of 2 file descriptors (int)



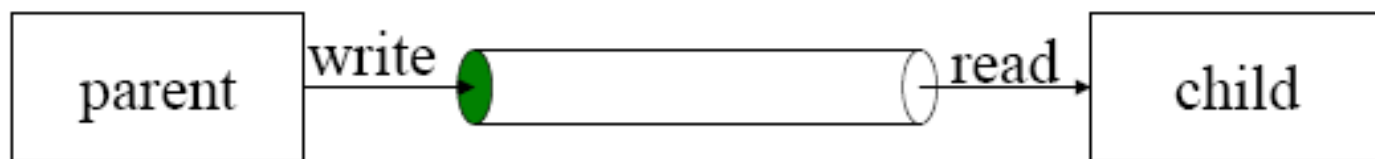


Pipe Example

```
int pid, p[2];
...
if (pipe(p) == -1)
    exit(1);
pid = fork();
if (pid == 0) {
    close(p[1]);
    ... read using p[0] as fd until EOF ...
}
else {
    close(p[0]);
    ... write using p[1] as fd ...
    close(p[1]); /* sends EOF to reader */
    wait(&status);
}
```

child

parent



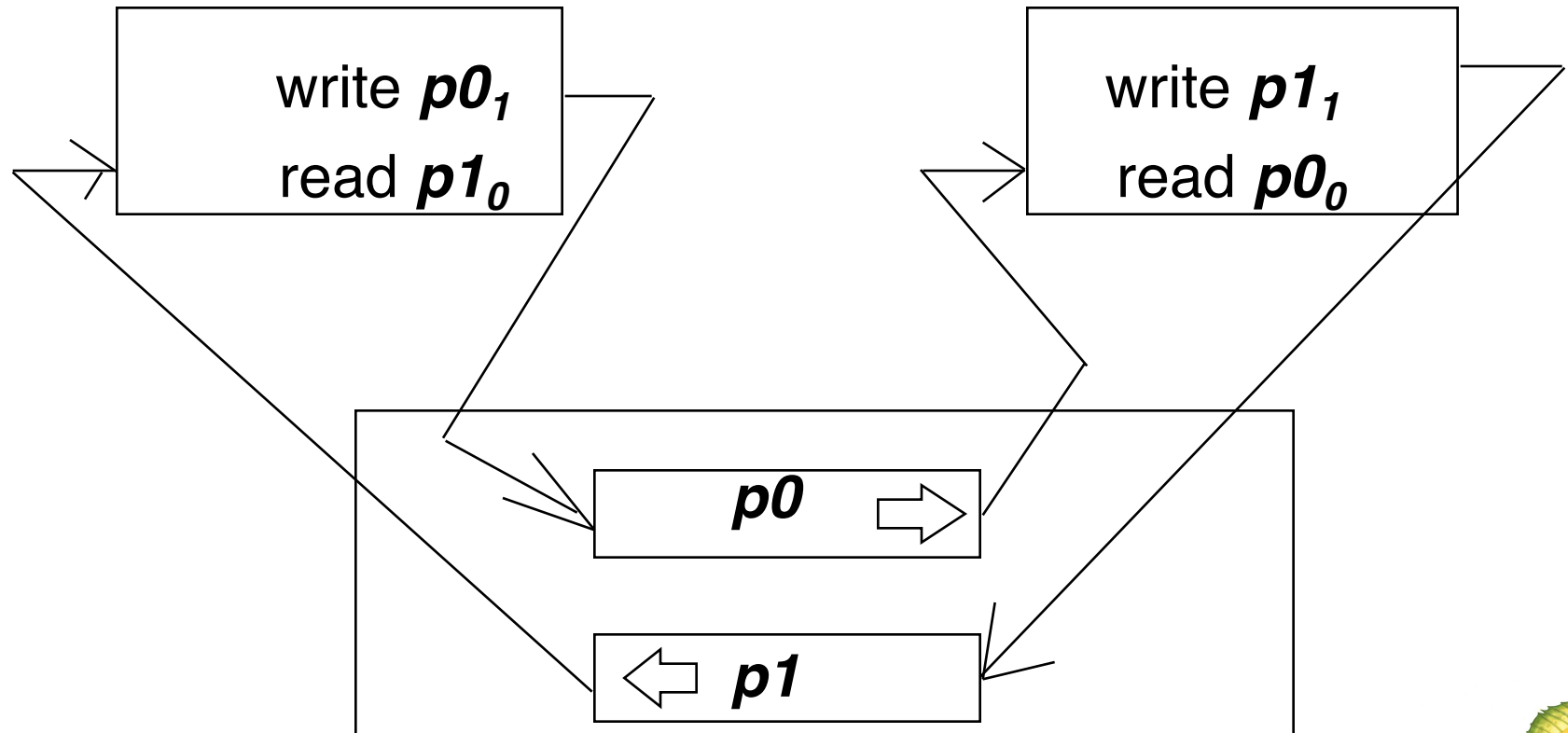


Full Duplex Channel via Two Pipes

Two separate pipes, say $p0$ and $p1$

Process A

Process B





write System Call

Function:

- To write ***nbytes*** to the write end of a pipe.
 - If a write process attempts to ***write*** to a full pipe, the default action is for the system to block the process until the data is able to be received.
- Include(s): `<unistd.h>`
- Syntax: ***ssize_t write (int fd, const void *buf, size_t nbytes);***
 - just the *write* system call
- Returns
 - success: Number of bytes written; Failure; -1; Sets errno:Yes.
- Arguments
 - ***int fd***: file descriptor;
 - ***const void *buf***: buffer;
 - ***size_t nbytes***: number of bytes in buffer





read System Call

Function:

- To read ***nbytes*** from the read end of a pipe.
 - if ***read*** is attempted on an empty pipe, the process will block until data is available.
- Includes: `<unistd.h>` `<sys/types.h>` `<sys/uio.h>`
- Syntax: ***ssize_t read(int fd, const void *buf, size_t nbytes);***
- Return
 - success: Number of bytes read;
 - Failure; -1; Sets errno:Yes.
 - EOF (0): write end of pipe closed
- Arguments
 - ***int fildes***: file descriptor;
 - ***const void *buf***: buffer;
 - ***size_t nbyte***: number of bytes





Hands-on Practice

- Question 4 in Project 1
- Code structure (pseudocode):

```
pid = fork();
int pipe[2];
if (pid < 0)
    error....;
else if (pid == 0)
    read from pipe[0];
    MyCompress();
    write to destination txt;
else
    read from source txt;
    write to pipe[1];
```

Or

```
pid = fork();
int pipe[2];
if (pid < 0)
    error....;
else if (pid == 0)
    read from pipe[0];
    write to destination txt;
else
    read from source txt;
    MyCompress();
    write to pipe[1];
```

